

Introdução a Algoritmos

Aula 1 — Introdução

Raoni F. S. Teixeira · 1s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Explicar o que é um algoritmo e distingui-lo de um programa de computador;
2. Descrever os três tipos de erro (sintaxe, execução e lógica) e identificar a qual categoria um erro dado pertence;
3. Escrever e compilar um programa mínimo em C, corrigindo erros de compilação de forma autônoma;
4. Aplicar as etapas do processo de resolução de problemas (compreender, planejar, implementar, verificar) a um problema simples.

1 Programar é montar um quebra-cabeça

Imagine que você precisa ensinar alguém a fazer um bolo — mas essa pessoa nunca entrou numa cozinha e segue instruções ao pé da letra, sem improviso, sem bom senso, sem intuição. Você não pode dizer “asse até ficar pronto”: precisa dizer exatamente quanto tempo, a que temperatura, em que posição do forno. Essa é a essência de programar: comunicar uma solução de forma precisa o suficiente para que uma máquina — que não adivinha intenções — possa executá-la corretamente.

O resultado dessa comunicação é um programa: uma sequência de instruções escritas em uma linguagem que o computador compreende. Mas antes de escrever qualquer programa, precisamos ter a solução clara em nossa cabeça. Essa solução é chamada de algoritmo.

Ao longo deste curso, você aprenderá a montar soluções como quem monta um quebra-cabeça. As peças do quebra-cabeça já existem — são um conjunto pequeno e bem definido de instruções que toda linguagem de programação oferece. O desafio não é inventar peças novas: é aprender a encaixá-las na ordem certa para resolver cada problema.

As peças fundamentais que você dominará neste curso são:

Peça	O que faz
Dados	Armazena informação na memória — desde um valor simples até coleções e registros compostos
Sequência	Executa instruções uma após a outra, em ordem
Decisão	Escolhe um caminho entre dois ou mais, dependendo de uma condição
Repetição	Repete um bloco de instruções enquanto uma condição for verdadeira
Função	Nomeia e reutiliza um bloco de instruções em vários lugares

Cada aula deste curso apresenta uma dessas peças em detalhe. No final, você será capaz de combinar todas elas para resolver problemas reais.

“Dados” é a peça mais rica do curso: variável, vetor, matriz, struct e ponteiro são todos a mesma peça — guardar e organizar informação na memória — em níveis crescentes de complexidade. A jornada de variável até ponteiro é a jornada de entender essa peça em toda a sua profundidade.

2 O que é um computador

A palavra computador vem de computar, que significa calcular. Antes das máquinas modernas, “computadores” eram pessoas contratadas para executar cálculos repetitivos seguindo instruções precisas — às vezes por horas, às vezes por dias.

Hoje, um computador é uma máquina capaz de realizar, em alta velocidade, operações matemáticas e lógicas sobre dados de entrada, produzindo resultados úteis. Sua essência é o processamento de informações, mediado por dois componentes centrais:

- Hardware — a parte física: processador, memória, dispositivos de entrada e saída.
- Software — o conjunto de instruções que dá vida ao hardware.

Internamente, a linguagem do computador é binária: tudo é representado por combinações de zeros e uns (bits). O agrupamento de 8 bits forma um byte, unidade capaz de representar um caractere, número ou símbolo.



Figure 1: Hierarquia de abstrações em um sistema computacional. Cada camada esconde os detalhes da camada inferior, simplificando o controle do sistema.

3 Algoritmos

Definição — Algoritmo

Um algoritmo é uma sequência finita, ordenada e não ambígua de passos que, quando executada, resolve um problema ou produz um resultado esperado.

A palavra vem do nome do matemático persa Al-Khwarizmi (século IX), cujos escritos introduziram o sistema de numeração decimal no mundo ocidental. O conceito, porém, é mais antigo: receitas culinárias, instruções de montagem e métodos de cálculo manual são todos algoritmos.

Um algoritmo precisa ser:

- Finito — deve terminar após um número finito de passos.
- Determinístico — dado o mesmo estado inicial, sempre produz o mesmo resultado.

- Executável — cada passo deve ser suficientemente simples para ser realizado.

Exemplo — Multiplicação à mão

Multiplique 2345×4567 usando lápis e papel. O método que você aprendeu na escola — multiplicar dígito por dígito e somar os resultados deslocados — é um algoritmo. Ele é finito (termina), determinístico (sempre dá o mesmo resultado) e executável (qualquer pessoa treinada consegue realizá-lo).

3.1 Do problema à solução

Um problema computacional define uma relação entre uma entrada e uma saída. Para construir um algoritmo, seguimos uma estratégia de quatro etapas inspiradas no método de George Pólya:

- Pa. Compreender o problema. É impossível resolver o que não se entende. Leia com atenção. Identifique claramente qual é a entrada e qual é a saída esperada.
- Pb. Elaborar um plano. Busque conexões entre os dados e a solução. Muitas vezes a resposta surge ao resolvermos um problema mais simples e relacionado. Esboce o algoritmo em linguagem natural antes de escrever código.
- Pc. Executar o plano. Coloque o algoritmo em ação, passo a passo. Simule manualmente com um exemplo pequeno antes de confiar na implementação.
- Pd. Avaliar a solução. Verifique se a resposta faz sentido para entradas variadas, incluindo casos extremos (zero, negativo, lista vazia). Se algo estiver errado, volte ao início.

Este processo não é linear: é comum perceber na etapa 3 que o plano da etapa 2 estava incompleto, e voltar atrás. Isso não é falha — é parte normal do raciocínio algorítmico.

3.2 Testar não é provar

Você acabou de ver o algoritmo do par/ímpar. Antes de continuar, responda mentalmente: o programa está correto?

Quase certamente você pensou em testar alguns valores — talvez 4, 7, 0 e -3. Todos funcionam. Mas isso prova que o programa está correto?

Não.

Para provar que $n \% 2 == 0$ identifica corretamente todos os números pares, você precisaria testar **todos** os inteiros — e há infinitos deles. Nenhuma quantidade finita de testes cobre esse espaço. O que você fez foi acumular **evidências**, não uma **prova**.

Isso não é uma limitação prática que a tecnologia resolve. É uma limitação de princípio.

Contexto

Immanuel Kant (1724–1804) distinguiu dois tipos de conhecimento: o **a posteriori**, que vem da experiência (dos testes, das observações), e o **a priori**, que precede a experiência e independe dela. Para Kant, a lógica é um conhecimento a priori — suas leis não são descobertas observando o mundo, mas derivadas da própria estrutura do raciocínio.

Em programação, isso tem uma consequência direta. Quando dizemos que $n \% 2 == 0$ é correto para **todo** inteiro par, não estamos generalizando testes — estamos aplicando uma propriedade matemática da divisão. A correção vem do raciocínio, não da experiência. A lógica booleana — AND, OR, NOT — é a estrutura a priori da computação: ela não foi descoberta empiricamente, ela **precede e torna possível** qualquer teste.

O compilador confirma isso a seu modo: ele verifica se o código segue as regras da linguagem — mas não verifica se o raciocínio está correto. Você pode escrever código sintaticamente perfeito que faz a coisa errada. O compilador aprovará sem hesitar.

Testes encontram bugs. A ausência de bugs nos testes não garante correção. Essa distinção — entre evidência e prova — acompanhará você por toda a carreira. Os melhores programadores não confundem “passou nos testes” com “está correto”.

3.3 Observar, induzir, generalizar

Se testar não é provar, como descobrimos algoritmos?

Na prática, observando padrões. Alguém percebeu que o resto de n dividido por 2 sempre é 0 para números pares e 1 para ímpares. Esse padrão foi observado, generalizado e formalizado como regra. É assim que a maioria do conhecimento algorítmico é construída — não por dedução pura, mas por **indução**: observar casos particulares e inferir uma regra geral.

Contexto

David Hume (1711–1776) identificou um problema fundamental nesse processo. Considere: você observa o sol nascendo todos os dias da sua vida. Você conclui que o sol nascerá amanhã. Mas essa conclusão é logicamente garantida pelas observações passadas? Não — nenhuma quantidade de casos particulares **prova** a regra geral. Isso é o **problema da indução**: a generalização vai além do que a experiência pode justificar.

Hume não diz que a indução é inútil — diz que ela é **não demonstrativa**. Usamos indução porque funciona, não porque provamos que funciona. É uma aposta racional, não uma certeza lógica.

Em programação, o problema da indução reaparece toda vez que você testa um programa:

Exemplo — O frango e o programador

Um frango é alimentado pelo fazendeiro todos os dias às 8 da manhã. Após 300 dias, o frango induziu com confiança: “o fazendeiro aparece às 8 para me alimentar”. No 301º dia, o fazendeiro aparece às 8 — mas não para alimentar o frango.

Um programa que “passa em todos os testes” pode estar na mesma situação. Os testes cobrem os casos que você pensou em testar. O mundo real providenciará os casos que você não pensou.

A lição não é desistir de testar — é entender o que o teste garante e o que não garante. Testes são **necessários** mas não **suficientes**. Eles aumentam nossa confiança; não produzem certeza.

A tensão entre Hume e Kant define dois modos de trabalho em computação: testamos empiricamente para ganhar confiança (Hume), mas provamos formalmente para ter certeza (Kant). A maioria dos programas vive no território de Hume. Sistemas críticos — software de aviação, equipamentos médicos, reatores nucleares — aspiram ao território de Kant, usando técnicas de **verificação formal**.

4 Das ideias aos programas: a ponte entre algoritmo e código

Considere o seguinte problema: dado um número inteiro, dizer se ele é par ou ímpar.

Podemos descrever a solução em linguagem natural:

Algoritmo (em português):

1. Leia um número inteiro n .
2. Calcule o resto da divisão de n por 2.
3. Se o resto for zero, escreva “par”. Caso contrário, escreva “ímpar”.

Esse algoritmo já tem tudo que precisamos: entrada (o número), processamento (o cálculo do resto) e saída (a mensagem). Agora só precisamos traduzir cada passo para uma linguagem que o computador entenda.

Em C — a linguagem que usaremos neste curso — a tradução fica assim:

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);           // Passo 1: leia n
    if (n % 2 == 0)             // Passo 2 e 3: resto e decisão
        printf("par\n");
    else
        printf("impar\n");
    return 0;
}
```

Observe que a estrutura do código espelha diretamente os passos do algoritmo. Isso não é coincidência: um bom programa é sempre a tradução fiel de um bom algoritmo. Inversamente, quando um programa é difícil de entender, quase sempre é porque o algoritmo subjacente não estava bem formulado.

Regra de ouro: nunca comece a digitar código sem antes ter o algoritmo claro. Programar sem algoritmo é como construir uma casa sem planta — você até pode chegar lá, mas o caminho será muito mais longo e o resultado, muito menos sólido.

5 A linguagem C

A linguagem C foi criada na década de 1970 por Dennis Ritchie nos laboratórios Bell, originalmente para escrever o sistema operacional Unix. Desde então, tornou-se uma das linguagens mais influentes da história da computação: C inspirou diretamente C++, Java, C# e Go, e ainda hoje é usada em sistemas embarcados, sistemas operacionais e aplicações de alto desempenho.

Por que aprender C num curso introdutório? Porque C expõe o funcionamento interno do computador sem escondê-lo atrás de abstrações excessivas. Ao aprender C, você entende por que as coisas funcionam como funcionam — e isso torna mais fácil aprender qualquer outra linguagem depois.

5.1 Estrutura de um programa C

Todo programa em C segue uma estrutura mínima:

```
#include <stdio.h>

int main() {
    /* instruções aqui */
    return 0;
}
```

Vamos entender cada parte:

Trecho	O que significa
<code>#include <stdio.h></code>	Inclui a biblioteca de entrada/saída padrão. Sem ela, não podemos usar <code>printf</code> nem <code>scanf</code> . Pense nisso como “trazer uma caixa de ferramentas” antes de começar o trabalho.
<code>int main()</code>	Define a função principal — o ponto de entrada do programa. Todo programa C começa sua execução aqui. O <code>int</code> indica que a função devolve um número inteiro ao sistema operacional.
<code>{ ... }</code>	As chaves delimitam o bloco de instruções da função. Tudo que está entre elas faz parte do <code>main</code> .
<code>return 0;</code>	Encerra o programa e devolve o valor 0 ao sistema operacional. Por convenção, 0 significa “programa encerrado com sucesso”. Qualquer outro valor indica erro.

5.2 O programa mais simples: Hello, World!

```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

Esse programa faz exatamente uma coisa: imprime a mensagem “Hello, world!” na tela. Apesar de simples, ele exercita toda a estrutura que acabamos de estudar — e por isso é, há décadas, o primeiro programa que todo programador escreve.

Para executá-lo, é necessário compilar o código-fonte, transformando-o em um arquivo executável:

```
$ gcc hello.c -o hello
$ ./hello
Hello, world!
```

O programa `gcc` é o compilador: ele lê o arquivo de texto (`hello.c`) e produz um arquivo executável (`hello`) que o processador consegue rodar diretamente.

5.3 Algoritmos revelam o que a intuição esconde

Uma das propriedades mais surpreendentes dos algoritmos é que eles frequentemente revelam verdades que o raciocínio intuitivo erra. Considere dois exemplos.

5.3.1 O paradoxo do aniversário

Em uma sala com 23 pessoas, qual a probabilidade de que pelo menos duas façam aniversário no mesmo dia do ano?

A intuição de quase todo mundo diz: pequena. Há 365 dias no ano e apenas 23 pessoas — parece que a chance de colisão é baixa.

O algoritmo discorda. A probabilidade de que **nenhuma** colisão ocorra é:

$$P(\text{sem colisão}) = \frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \dots \times \frac{343}{365}$$

Calculando esse produto:

```
#include <stdio.h>
int main() {
    double p = 1.0;
    for (int k = 1; k < 23; k++)
        p *= (365.0 - k) / 365.0;
    printf("P(sem colisao) = %.4f\n", p);
    printf("P(colisao)      = %.4f\n", 1 - p);
    return 0;
}
```

Saída: P(colisao) = 0.5073

Com apenas 23 pessoas, a probabilidade de colisão supera 50%. Com 50 pessoas, supera 97%. A intuição erra porque subestima o número de **pares** possíveis: 23 pessoas formam $23 \times \frac{22}{2} = 253$ pares — e cada par tem sua chance de colisão.

O programa não inventou esse resultado: ele **calculou** o que já estava implícito na matemática, mas que nosso julgamento intuitivo não conseguia enxergar.

5.3.2 O tabuleiro de xadrez

Um grão de trigo na primeira casa, dois na segunda, quatro na terceira, e assim por diante — dobrando a cada casa. Quantos grãos na 64ª casa?

```
#include <stdio.h>
int main() {
    unsigned long long graos = 1;
    unsigned long long total = 0;
    for (int casa = 1; casa <= 64; casa++) {
        total += graos;
        printf("Casa %2d: %llu\n", casa, graos);
        graos *= 2;
    }
    printf("Total: %llu\n", total);
    return 0;
}
```

A 64ª casa contém $2^{63} \approx 9 \times 10^{18}$ grãos — mais do que toda a produção mundial de trigo nos últimos mil anos. O processo de duplicação é simples de **descrever**; seu resultado é impossível de **prever** sem calcular.

Algoritmos são ferramentas de descoberta, não apenas de execução. Eles tornam acessíveis consequências que estavam implícitas nos dados mas além do alcance da intuição. Parte do poder da programação está exatamente aqui: transformar perguntas difíceis em computações executáveis.

5.4 Os limites do computável

Vimos que algoritmos são poderosos — resolvem problemas, revelam verdades, automatizam raciocínios complexos. Mas há algo que nenhum algoritmo pode fazer?

Sim. E a prova disso é uma das grandes conquistas intelectuais do século XX.

5.4.1 O Teorema da Parada

Em 1936 — antes que existisse qualquer computador moderno — o matemático britânico Alan Turing provou que é impossível construir um algoritmo geral que decida, para **qualquer** programa e **qualquer** entrada, se esse programa vai eventualmente parar ou entrar em loop infinito.

Isso é o **Teorema da Parada (Halting Problem)**.

Para sentir por que ele é verdadeiro, comece com um exemplo concreto. Considere o seguinte programa:

```
/* Conjectura de Goldbach: todo inteiro par > 2 e
   soma de dois numeros primos. Exemplos:
   4 = 2+2, 6 = 3+3, 8 = 3+5, 100 = 3+97, ...
   Ninguém provou (nem refutou) isso para todos os pares. */

int main() {
    for (long long n = 4; ; n += 2) {
        if (!e_soma_de_dois_primos(n)) {
            printf("Goldbach e falso! Contraexemplo: %lld\n", n);
            return 0;
        }
    }
}
```

Este programa para **se e somente se** a Conjectura de Goldbach for falsa. Se a conjectura for verdadeira (como todos acreditam mas ninguém provou), o programa roda para sempre.

Perguntar “esse programa vai parar?” é equivalente a resolver um dos problemas abertos mais famosos da matemática.

5.4.2 A prova por contradição

Mas Turing foi além: ele provou que o problema da parada é **indecidível para qualquer programa**, não apenas para esse. A prova funciona por contradição — e tem a elegância de um paradoxo.

Suponha que exista um programa `termina(P)` que recebe qualquer programa `P` como entrada e devolve 1 se `P` para e 0 se `P` nunca para. Com esse programa em mãos, construa o seguinte programa `D`:

```
programa D:
  se termina(D) == 1: /* se D supostamente para */
    repita para sempre /* entao D NAO para */
  senao: /* se D supostamente nao para */
    pare /* entao D PARA */
```

Agora pergunte: `D` para?

- Se `termina(D)` diz que `D` para (retorna 1): então `D` entra em loop infinito. Mas `termina` disse que `D` para — contradição.
- Se `termina(D)` diz que `D` não para (retorna 0): então `D` para imediatamente. Mas `termina` disse que `D` não para — contradição.

Em qualquer caso, `termina` está errado sobre `D`. Logo, um programa `termina` perfeito não pode existir.

A estrutura desse argumento é a mesma do paradoxo do mentiroso (“esta frase é falsa”): a **autorreferência** cria uma contradição irresolúvel. Turing transformou esse paradoxo filosófico numa prova matemática rigorosa.

5.4.3 O que isso significa na prática

O Teorema da Parada não diz que **nenhum** programa específico pode ter sua parada verificada — muitos podem. Diz que não existe um **método geral** que funcione para todos os programas.

As consequências são profundas:

- Não existe verificador de vírus perfeito: decidir se um programa arbitrário tem comportamento malicioso é equivalente ao problema da parada.
- Não existe compilador que detecta todos os loops infinitos: compiladores podem detectar alguns casos óbvios, mas não todos.
- Alguns problemas matemáticos podem ser genuinamente incomputáveis: não por falta de tecnologia, mas por razão matemática fundamental.

Contexto

Alan Turing (1912–1954) provou o Teorema da Parada em 1936, antes de qualquer computador moderno existir, usando o conceito abstrato de **Máquina de Turing**. O mesmo artigo que continha essa prova de impossibilidade também definia formalmente o que significa **computar** — e é considerado o documento fundador da ciência da computação.

O resultado de Turing está em diálogo direto com o Teorema da Incompletude de Gödel (1931), que provou que em qualquer sistema formal suficientemente expressivo existem afirmações verdadeiras que não podem ser provadas dentro do sistema. Hume estava certo que a indução não prova tudo; Gödel e Turing mostraram que a dedução também tem limites.

Começamos esta aula dizendo que programar é montar um quebra-cabeça com um conjunto fixo de peças. O Teorema da Parada revela algo mais profundo: há quebra-cabeças para os quais as peças simplesmente não existem — não porque ninguém as inventou, mas porque a lógica proíbe sua existência.

Isso não é motivo de desânimo. É, ao contrário, uma das conquistas intelectuais mais belas da humanidade: a demonstração, usando apenas raciocínio, dos limites exatos do raciocínio.

6 Erros: parte esperada do processo

Todo programador, do iniciante ao experiente, comete erros. Reconhecer isso desde o início é importante: um erro não é sinal de incompetência, mas uma informação sobre o que precisa ser ajustado. O processo de encontrar e corrigir erros chama-se depuração (do inglês debugging).

Existem três categorias principais de erros:

Tipo	Quando ocorre	Exemplo
Sintaxe	O compilador detecta antes de gerar o executável. O programa nem chega a rodar.	Esquecer o ; no final de uma instrução.
Execução	O programa compila, mas trava ou encerra com erro durante a execução.	Dividir um número por zero.
Lógica	O programa roda e termina, mas produz um resultado incorreto.	Usar > quando deveria ser >=, fazendo o programa perder um caso.

Atenção

Erros de lógica são os mais difíceis de encontrar, porque o compilador não os detecta — o programa “parece funcionar”. A única defesa é testar o programa com exemplos variados, incluindo casos extremos.

7 Exemplo prático

O programa abaixo lê dois números e exibe o maior deles. Antes de ler o código, tente escrever o algoritmo em português usando as quatro etapas da Seção 3.

```
#include <stdio.h>

int main() {
    int x, y;

    /* Entrada */
    printf("x: ");
    scanf("%d", &x);
    printf("y: ");
    scanf("%d", &y);

    /* Processamento e saída */
    if (x > y)
        printf("O maior numero e x = %d\n", x);
    else
        printf("O maior numero e y = %d\n", y);

    return 0;
}
```

Perceba que o programa está dividido em duas partes claramente separadas pelos comentários: a entrada de dados e o processamento com saída. Manter essa separação explícita é um bom hábito — ela torna o código mais fácil de ler e modificar.

Atenção

Este programa tem um defeito sutil: e se x e y forem iguais? O que ele imprime? O que deveria imprimir? Esse é um exemplo de erro de lógica — identifique-o e corrija-o como exercício.

8 Conclusão

Nesta aula, estabelecemos os fundamentos sobre os quais todo o curso será construído.

Os pontos centrais são:

1. Programar é traduzir algoritmos em código. Antes de qualquer linha de C, é preciso ter a solução clara — e a melhor forma de clarificar uma solução é escrevê-la em linguagem natural, passo a passo.
2. Algoritmos são montados com um conjunto pequeno de peças. Variáveis, sequências, decisões, repetições e funções são suficientes para descrever qualquer computação. Nas próximas aulas, estudaremos cada uma dessas peças em detalhe.
3. Erros são parte do processo. Nenhum programa nasce correto. A habilidade de ler mensagens de erro, testar com exemplos e raciocinar sobre o comportamento do programa é tão importante quanto conhecer a sintaxe da linguagem.

8.1 Referências

- Ritchie, D. M. & Kernighan, B. W. The C Programming Language. Prentice Hall, 1978.
- Pólya, G. How to Solve It. Princeton University Press, 1945.